In [1]:
```
# a. Import necessary packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense

# b. Load and preprocess the image data
train = pd.read_csv('sign_mnist_train.csv')
test = pd.read_csv('sign_mnist_test.csv')

# Separate labels and images
X_train = train.drop('label', axis=1).values
y_train = train['label'].values

X_test = test.drop('label', axis=1).values
y_test = test['label'].values

# Normalize pixel values
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

# Reshape images into 28x28 format
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# c. Define the CNN architecture
model = Sequential([
    Input(shape=(28, 28, 1)),

    Conv2D(32, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),

    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dense(25, activation='softmax')
])

# Compile the model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

# d. Train the model
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.1
```

```
)

# e. Estimate model performance
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")

# Predict one sample image
prediction = model.predict(X_test[:1])

print("Predicted Sign:", np.argmax(prediction[0]))
print("Actual Sign:", y_test[0])
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Pleas
e use WSL2 or the TensorFlow-DirectML plugin.
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 25)	3,225

Total params: 226,969 (886.60 KB)

Trainable params: 226,969 (886.60 KB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
387/387 ━━━━━━━━━━━━━━━━━━━━ 13s 22ms/step - accuracy: 0.6438 - loss: 1.2041 - va
l_accuracy: 0.9195 - val_loss: 0.2847
Epoch 2/10
387/387 ━━━━━━━━━━━━━━━━━━━━ 8s 19ms/step - accuracy: 0.9672 - loss: 0.1400 - val
_accuracy: 0.9964 - val_loss: 0.0474
Epoch 3/10
387/387 ━━━━━━━━━━━━━━━━━━━━ 9s 22ms/step - accuracy: 0.9983 - loss: 0.0260 - val
_accuracy: 0.9924 - val_loss: 0.0358
Epoch 4/10
387/387 ━━━━━━━━━━━━━━━━━━━━ 7s 18ms/step - accuracy: 0.9996 - loss: 0.0087 - val
_accuracy: 0.9978 - val_loss: 0.0128
Epoch 5/10
387/387 ━━━━━━━━━━━━━━━━━━━━ 6s 15ms/step - accuracy: 0.9994 - loss: 0.0061 - val
_accuracy: 1.0000 - val_loss: 0.0028
Epoch 6/10
387/387 ━━━━━━━━━━━━━━━━━━━━ 6s 15ms/step - accuracy: 1.0000 - loss: 0.0021 - val
_accuracy: 1.0000 - val_loss: 0.0020
Epoch 7/10
387/387 ━━━━━━━━━━━━━━━━━━━━ 6s 15ms/step - accuracy: 1.0000 - loss: 0.0011 - val
_accuracy: 1.0000 - val_loss: 0.0020
Epoch 8/10
387/387 ━━━━━━━━━━━━━━━━━━━━ 6s 15ms/step - accuracy: 0.9903 - loss: 0.0300 - val
_accuracy: 1.0000 - val_loss: 8.6433e-04
Epoch 9/10
387/387 ━━━━━━━━━━━━━━━━━━━━ 6s 14ms/step - accuracy: 1.0000 - loss: 5.9616e-04 -
val_accuracy: 1.0000 - val_loss: 5.5705e-04
Epoch 10/10
387/387 ━━━━━━━━━━━━━━━━━━━━ 6s 14ms/step - accuracy: 1.0000 - loss: 3.7673e-04 -
val_accuracy: 1.0000 - val_loss: 4.2239e-04
225/225 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9076 - loss: 0.4111
Test Loss: 0.4111042320728302
Test Accuracy: 90.75571894645691 %
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 166ms/step
Predicted Sign: 6
Actual Sign: 6
```

In [ ]: